

Sang Ah Lee
DATA-260 Section 22
Student ID: 012634544

Configuration:

SID4: 4544
PORT_BASE: 8044
PREFIX: s4544
SEED: 4544
VERIFY_SEED: 264544
DOMAIN_ID: 4 (Open_source package vulnerabilities)

Hardware: Apple M3, 16GB

Model: qwen2.5:3b (switched from qwen3.8b due to hardware issues)

Recent repo state (w/o report): e51f11d3bd94583867b7ac8bab66d98c0105ab55

Link: <https://github.com/sangahlee0/data260-4544>

Link to HW3 Commit#:

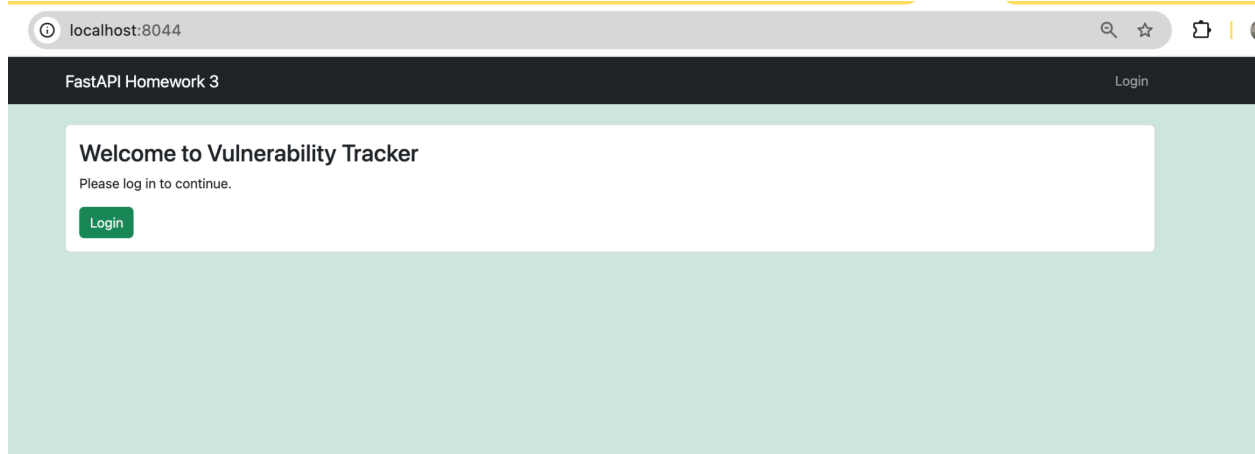
<https://github.com/sangahlee0/data260-4544/tree/e51f11d3bd94583867b7ac8bab66d98c0105ab55> (This commit link does not include the report, because it will be added after downloading this document)

Homework 3

1. Routes - Handled Separately in a router:

- Home Page (/): home.html

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="/">FastAPI Homework 3</a>
    <div class="navbar-nav">
      {% if user %}
        <a class="nav-link" href="/dashboard">Dashboard</a>
        <a class="nav-link" href="/logout">Logout</a>
      {% else %}
        <a class="nav-link" href="/login">Login</a>
      {% endif %}
    </div>
  </div>
</nav>
```



- Login Page (/login): login.html

```
<title>FastAPI Homework 3 - Login</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap"
</head>

<body>

  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand" href="/">FastAPI Homework 3</a>
      <div class="navbar-nav">
        {% if user %}
          <a class="nav-link" href="/dashboard">Dashboard</a>
          <a class="nav-link" href="/logout">Logout</a>
        {% else %}
          <a class="nav-link active" href="/login">Login</a>
        {% endif %}
      </div>
    </div>
  </nav>
```

FastAPI Homework 3

Login

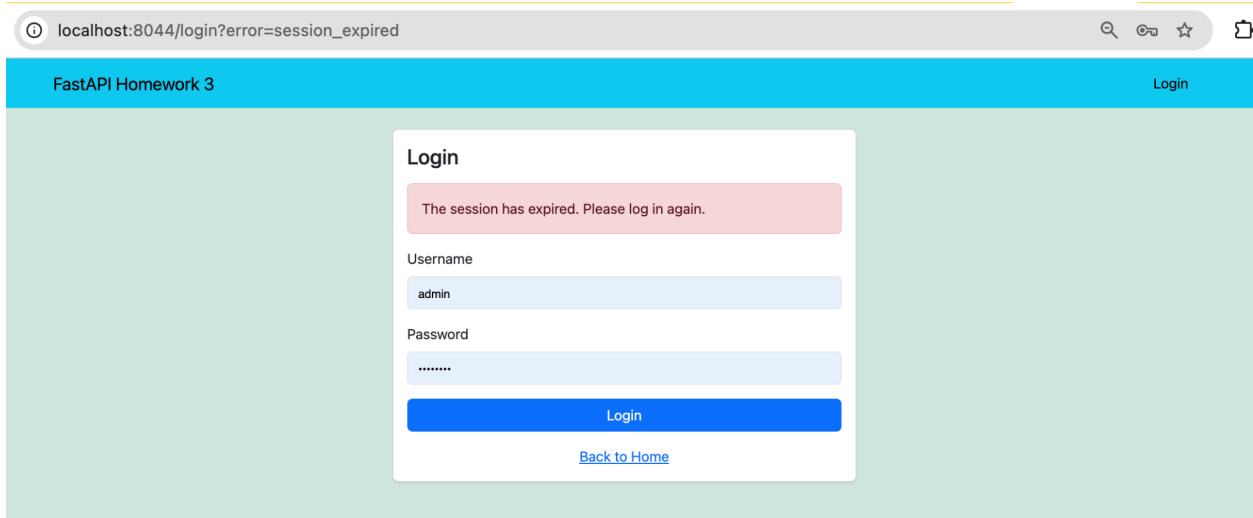
Login

Username

Password

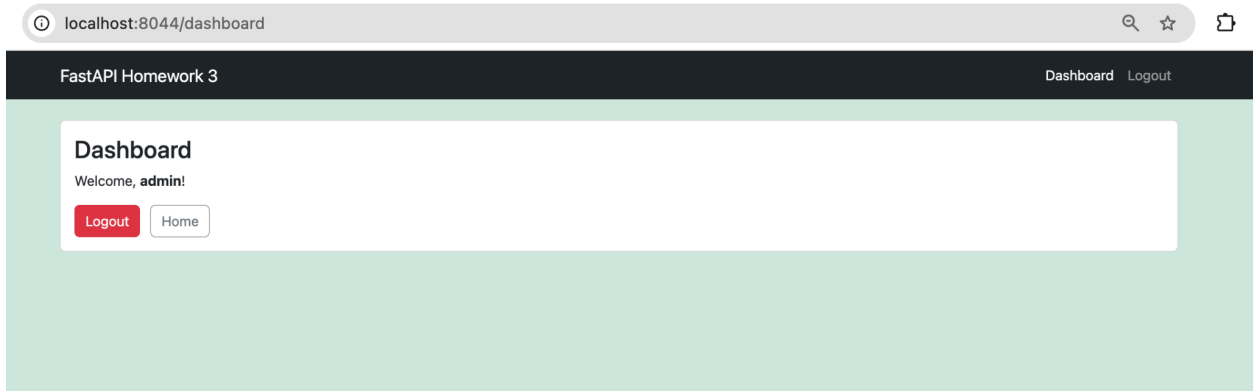
Login

[Back to Home](#)



- Dashboard Page (/dashboard): dashboard.html

```
0
1 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
2   <div class="container">
3     <a class="navbar-brand" href="/">FastAPI Homework 3</a>
4     <div class="navbar-nav">
5       <a class="nav-link active" href="/dashboard">Dashboard</a>
6       <a class="nav-link" href="/logout">Logout</a>
7     </div>
8   </div>
9 </nav>
10
11 <div class="container mt-4">
12   <div class="card">
13     <div class="card-body">
14       <h3 class="card-title">Dashboard</h3>
15       <p class="card-text">Welcome, <strong>{{ user }}</strong>!</p>
16
17       <a class="btn btn-danger" href="/logout">Logout</a>
```



- Logout (/logout):

DashboardWelcome, **admin**!

Logout

Home

2. Session Management:

- Store the logged-in user's information in the session

```
@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        # Store logged-in user in session
        request.session["user"] = username

        # Redirect user to dashboard
        return RedirectResponse(
            url="/dashboard",
            status_code=HTTP_302_FOUND
        )
```

- Session cookie is secure

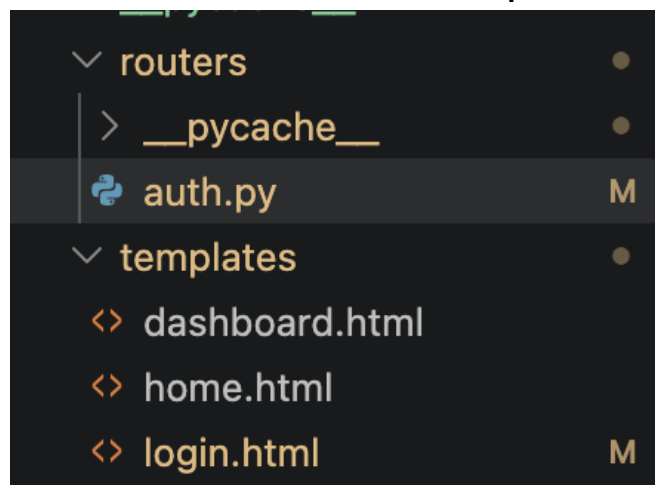
```
# Enable session support
app.add_middleware(
    SessionMiddleware,
    secret_key=SECRET_KEY,
    https_only=True,
    same_site="lax",
    max_age=3600
)
```

```
[ % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   100      32     0     0   100      32      0     617  --:--:--  --:--:--  --:--:--   627
HTTP/1.1 302 Found
date: Mon, 21 Sep 2026 03:44:51 GMT
server: uvicorn
content-length: 0
location: /dashboard
vary: Cookie
set-cookie: session=[REDACTED]; path=/; Max-Age=3600; httponly; samesite=lax; secure

(base) sangah@MacBook-Air-4 ~ %
```

3. Styling with Bootstrap

- Refer to Part 1 Problem 1 for pictures of the 3 pages



```

from fastapi import APIRouter, Request, Form
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
from starlette.status import HTTP_302_FOUND

import time

# Idle time-out
IDLE_TIMEOUT_SECONDS = 180 # 3 min inactivity

```

```

> web_application > routers > auth.py > ...
router = APIRouter()

# Configure Jinja2 templates directory
templates = Jinja2Templates(directory="templates")

# Hardcoded credentials for demo purposes only
# In real applications, credentials come from a database
VALID_USERNAME = "admin"
VALID_PASSWORD = "password"

@router.get("/")
def home(request: Request):
    """
    Home page route.

    - Checks if a user is logged in using the session
    - Passes user info to the template
    """
    user = request.session.get("user")

    return templates.TemplateResponse(
        # Call home for the home page html; we used index for main form page
        request,
        "home.html",
        {
            "request": request,
            "user": user
        }
    )

```

```
@router.get("/login")
def login_page(request: Request):
    """
    Displays the login form.

    If the user is already logged in,
    the template can choose what to display.
    """
    user = request.session.get("user")
    error = request.query_params.get("error") # Checks if there is somethig named error, otherw.

    if error == "session_expired":
        e_message = "The session has expired. Please log in again."
    elif error:
        e_message = "The username or password is invalid. Please try again."
    else:
        e_message = None

    return templates.TemplateResponse(
        request,
        "login.html",
        {
            "request": request,
            "user": user,
            "error": e_message
        }
    )
```

```

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        # Store logged-in user in session
        request.session["user"] = username
        # For the time where user is last active
        request.session["last_active"] = time.time()

        # Redirect user to dashboard
        return RedirectResponse(
            url="/dashboard",
            status_code=HTTP_302_FOUND
        )

    # If credentials are invalid:
    # Redirect back to login page
    #
    # NOTE:
    # No error message is shown intentionally.

    # Students are expected to add Bootstrap alerts.
    return RedirectResponse(
        url="/login?error=1", # Error response
        status_code=HTTP_302_FOUND
    )

```

```

@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected route.

    - Only accessible if user is logged in
    - Redirects to login page if session is missing
    """
    user = request.session.get("user")
    last_active = request.session.get("last_active") # For when user was last active

    # If user is not logged, block access
    if not user or not last_active:
        return RedirectResponse(
            url="/login",
            status_code=HTTP_302_FOUND
        )

    # If user is idle and it is past the active, block access
    if time.time() - last_active > IDLE_TIMEOUT_SECONDS:
        request.session.clear()
        return RedirectResponse(
            url="/login?error=session_expired",
            status_code=HTTP_302_FOUND
        )

    # Else if the user is active, then keep the time refreshed to most recent
    request.session["last_active"] = time.time()

    # If user is logged in, render dashboard
    return templates.TemplateResponse(
        request,
        "dashboard.html",
        {
            "request": request,
            "user": user
        }
    )

```

```

@router.get("/logout")
def logout(request: Request):
    """
    Logs the user out.

    - Clears all session data
    - Redirects back to home page
    """
    request.session.clear()

    return RedirectResponse(
        url="/",
        status_code=HTTP_302_FOUND
    )

```

Part 2: Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG)

A) Environment and Setup

```
((venv) ) (base) sangah@MacBook-Air-4 data260-4544 % pip install llama-index llama-index-embeddings-huggingface \
sentence-transformers faiss-cpu numpy pandas pyyaml
Downloading tqdm-4.66.1-py3-none-any.whl (20 kB)
Downloading tiktoken-0.14.0-cp312-cp312-macosx_11_0_arm64.whl (1.0 MB)
1.0/1.0 MB 33.0 MB/s 0:00:00
Downloading tinytag-2.3.2-py3-none-any.whl (37 kB)
Downloading torch-2.14.0-cp312-cp312-macosx_14_0_arm64.whl (127.3 MB)
127.3/127.3 MB 36.1 MB/s 0:00:03
Using cached sympy-1.14.0-py3-none-any.whl (6.3 MB)
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Downloading typing-inspect-0.9.0-py3-none-any.whl (8.8 kB)
```

B) One retrieval-only pipeline per technique

Chunking and Indexing

```
documents = load_documents()
embed_model = HuggingFaceEmbedding(model_name="sentence-transformers/all-MiniLM-L6-v2")

# Chunking
token_nodes = build_token_nodes(documents)
semantic_nodes = build_semantic_nodes(documents, embed_model)
sentence_window_nodes = build_sentence_window_nodes(documents)

# Indexing in-memory
token_index = build_index(token_nodes, embed_model)
semantic_index = build_index(semantic_nodes, embed_model)
sentence_window_index = build_index(sentence_window_nodes, embed_model)
```

Retrieval

```
##### Retrieval #####
def retriever_helper(index, technique, query, k, embed_model, qid="q1"):
    # Compute the query embedding
    query_embedding = np.array(embed_model.get_query_embedding(query))

    # Show its dimension and the first 8 values
    print("\n" + "=" * 75)
    print(f"\nTechnique: {technique}")
    print(f"Query vector shape: {query_embedding.shape}")
    print(f"Query dimension: {len(query_embedding)}")
    print(f"First 8 values: {query_embedding[:8]}")

    # Retrieve top-k nodes
    retriever = VectorIndexRetriever(index=index, similarity_top_k=k)

    # Start time
    start_time = time.perf_counter()

    results = retriever.retrieve(query)

    retrieval_latency_ms = (time.perf_counter() - start_time) * 1000
```

Report

1)

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@3 cosine	Recall@3	Mean retrieval latency (ms)
Token	3,615	542.61	0.6620	0.5671	0.80	43.74
Semantic	826	1,970.38	0.5892	0.4483	0.80	13.89
Sentence-window	13,165	123.63	0.6543	0.5478	1.00	126.77

2)

In this, the sentence-window technique performed better on this query. This may be due to the fact that the sentence-window technique reduces “hanging sentence fragments”, and keeps the sentences in-tact. By keeping the context together, it created a better technique for retrieving the correct source. As I am analyzing the correct source, sentence-window was the best, whereas token chunking shows the best mix of retrieval latency and chunks, reflective of how token chunking is the best for budget control.

3)

For my corpus, I believe that token chunking may be the best to utilize. The measurements show a good middle-ground for the chunks, chunk-length, the recall, and mean retrieval latency, compared to the other two (semantic and sentence-window) techniques. The mean @ k=3 cosine level was also the highest (0.5671), making it the best balance between the retrieval quality and speed.